

Chapter 1

Introduction to Recommender Systems

1.1 Why Personalization Matters

Think about your typical day. You might wake up to a playlist that Spotify assembled based on your listening habits. You scroll through a news feed that Instagram curated to maximize your engagement. You order groceries on Instacart, and the app surfaces products you are likely to buy based on your purchase history. Later, you search for a product on Amazon, and the results are not just sorted by relevance to your query, they are personalized to your browsing behavior, purchase patterns, and even the time of day. By the time you sit down in the evening to watch something on Netflix, the artwork for each title has been customized: the same movie might show a romance-themed poster to one viewer and an action-themed poster to another.

Personalization is the process of adapting a product experience to an individual user using information about that user, the available items, and the current context. A recommender system is the engineering system that performs this adaptation by predicting which items a user is most likely to find relevant or engage with. Because recommender systems are the primary mechanism for AI-powered personalization throughout this book, we use the terms recommender systems, RecSys, and personalization interchangeably unless a distinction is needed.

Recommender systems are among the most commercially impactful applications of machine learning in the consumer internet. Netflix reported that its recommender system influenced the choice of about 80 percent of hours streamed on the service (Gomez-Uribe and Hunt, 2015). More recent Netflix research continues to show that personalization affects both whether members watch and which titles they choose (Zielnicki et al., 2026). YouTube reported in 2018 that recommendations drove more than 70 percent of watch time on its platform (Nieva, 2018). In many consumer-facing products, the recommender system-enabled UI/UX is the core product.

For ML engineers who have built classification models, regression models, or even NLP pipelines, recommender systems can feel familiar. After all, ranking a set of items for a user looks a lot like a supervised learning problem. But as we will see throughout this book, the engineering challenges, data considerations, and system design tradeoffs in RecSys are substantially different from what is encountered in most traditional data science workflows.

This chapter sets the stage for the rest of the book. We will start with the core concepts that underpin all personalization systems, introduce a practical taxonomy of modeling approaches, explore what makes RecSys engineering distinct from conventional ML, and provide a roadmap for the chapters ahead.

1.2 Core Concepts in Personalization and RecSys

1.2.1 The Scoring Function: $f(\text{user}, \text{item}, \text{context})$

At its core, recommender systems try to answer a deceptively simple question: given a user, which items should we show them right now?

The simplest intuition if you are coming from another domain is that we can express the recommender system's behavior as a scoring function:

$$\text{Score} = f(\text{user}, \text{item}, \text{context})$$

Where f is some function; **user** represents anything we know about the person such as their browsing history, their past purchases, their embedding in some learned latent space, etc. **Item** represents everything we know about the candidate. If it is an ecommerce setting, then the item attributes could be its category, its price, its textual description, its popularity, its image embedding, etc. And **context** captures the situational factors such as the time of day, the device being used, where on the page the recommendations are displayed, geographic the location, etc..

The job of the recommender system is to evaluate this function for potentially millions or billions of candidate items and return the top-k that maximize some objective (engagement, revenue, satisfaction) within a strict latency budget often under few hundred milliseconds.

This framing highlights two challenges that set RecSys apart from typical ML problems: the scale of the candidate space and the real-time serving constraint. It is non-trivial to run a complex neural network on every item in a catalog of billions within in a reasonable latency. This tension between accuracy and latency is what drives the multi-stage architecture that we will discuss in detail in Chapter 2.

1.2.2 Brief History of Recsys: Collaborative vs Content-Based Filtering

Introductory ML courses that teach the history of the recommender systems, frame it as having been built on two foundational paradigms – collaborative filtering and content filtering. While the field has since moved on from referring to their implementations in terms of these techniques, it is still a good place to start to see how the thinking has evolved.

Collaborative Filtering (CF) operates on a simple intuition: users who have aligned interests in the past tend to also align in similar future situations. User A and User B both liked items 1, 2, and 3, and User A also liked item 4, then User B will probably like item 4 too. CF methods range from simple neighborhood-based approaches (finding the k most similar users or items) to matrix factorization techniques like SVD and ALS that decompose the user-item interaction matrix into low-dimensional latent factors.

Classical CF techniques relied on users' interactions with items rather than on descriptive item attributes. Early automated systems such as *GroupLens* predicted preferences from rating patterns, so the algorithm could operate without genre, text, or other item-content metadata (Resnick et al., 1994). This was not a rule across all of the earliest literature. *Tapestry*, which introduced the term collaborative filtering, stored document information together with user annotations (Goldberg et al., 1992). Collaborative methods still struggle with cold-start scenarios, and they can amplify popularity bias because frequently observed items dominate the interaction data.

Content-Based Filtering recommends items using their attributes and their similarity to items the target user has previously engaged with. If you watched three science fiction movies, for example, a content-based system might recommend more science fiction movies based on genre, director, or description similarity. Pandora's Music Genome Project is a content-driven example: trained music analysts annotate songs with harmony, rhythm, melody, vocals, instrumentation, lyrics, and other attributes that feed Pandora's recommendation algorithms (Pandora, 2022). Pandora's broader system has evolved beyond a single textbook method, so this example illustrates its content signal rather than classifying the entire service as purely content-based.

Content-based methods excel in cold-start scenarios for items (as long as metadata is available) but struggle with serendipity because they tend to over-specialize, trapping users in a “filter bubble” of similar content.

The distinction between collaborative and content-based filtering remains useful for understanding where recommendation signals originate, but modern systems routinely combine interactions, item features, context, and learned embeddings. Despite their differences, many early collaborative and content-based methods shared a computational pattern: represent users and items through interactions, attributes, or latent factors, then compare those representations using cosine similarity, a dot product, or another matching function. Some systems computed these relationships during serving, while others precomputed neighborhoods offline. Modern embedding-based retrieval preserves this core idea at much larger scale. User and item encoders produce dense representations, and approximate nearest-neighbor search identifies items whose representations best match the user representation. Chapters 5, 10, and 11 develop this connection in detail.

1.3 A Taxonomy of Modeling Approaches

One of the challenges for engineers entering the RecSys space is the sheer variety of modeling approaches available. To help build a mental map, we can organize the major families into five groups.

Group 1: Similarity and Content-Based Methods

This is where most teams historically used to start and these are the techniques you will see the most in most educational content from a decade on recsys. These methods personalize by finding items similar to what the user has previously engaged with. They include:

- **Content-Based Filtering:** Recommend items with similar metadata (genre, description, category) to the user's history. Requires good item metadata.
- **Neighborhood-Based Collaborative Filtering:** Find the most similar users (user-based CF) or most similar items (item-based CF) using co-occurrence patterns. Amazon's pioneering item-to-item CF (Linden et al., 2003) demonstrated that precomputing item similarities offline enables millisecond-level online serving.
- **Matrix Factorization:** Decompose the user-item interaction matrix into low-rank latent factors (SVD, ALS, NMF). The backbone of the Netflix Prize-era systems.

These methods are strong early baselines that require relatively simple infrastructure. Their common failure modes include echo chambers, popularity bias, and sparsity in the interaction matrix.

Group 2: Retrieval Engines

As catalogs grow beyond tens of thousands of items, brute-force scoring becomes infeasible. Retrieval engines solve this by learning dense vector representations of users and items, then using approximate nearest neighbor (ANN) search to find candidates efficiently. They include:

- **Two-Tower Neural Networks:** Separate encoders for user and item that project both into a shared embedding space. The user tower encodes user history and context; the item tower encodes item features. At serving time, item embeddings are pre-indexed (using FAISS, ScaNN, or similar), and only the user embedding needs to be computed in real time. YouTube's 2016 paper (Covington et al.) popularized this architecture.
- **Graph-Based Retrieval:** Methods like PinSage (Ying et al., 2018) that learn item embeddings from graph structures (e.g., user-item bipartite graphs) using graph neural networks.

In Chapter 5, we build two-tower retrieval models on the SIGIR E-commerce (Coveo) dataset. And in Chapter 10, we explore the item embedding strategies that underpin these retrieval systems, comparing pre-trained text encoders (SBERT), collaborative embeddings (Item2Vec), contrastive fine-tuning, and multi-modal fusion across four different datasets.

Group 3: Ranking Models

Once retrieval surfaces a manageable set of candidates (typically hundreds to low thousands), ranking models score and order them using richer features and more expressive models. They include a) tree-based models like XGBoost b) Deep models like DCN-V2 (Google), DLRM (Meta), DeepFM, Wide & Deep that learn complex feature interactions and c) Sequence models. The way they learn can be framed as follows:

- **Pointwise Rankers:** Predict a score (e.g., probability of click) for each item independently. Gradient boosted trees (XGBoost, LightGBM) are the workhorse here, and Learning-to-Rank (LTR) formulations optimize for ranking metrics directly.
- **Pairwise Rankers:** Predict which of two items should rank higher. RankNet (Burges et al., 2005) introduced this paradigm, later refined by LambdaMART.
- **Listwise Rankers:** Optimize the entire ranked list directly against a metric like NDCG.

In Chapter 6, we train XGBoost ranking models on the Yandex Yambda music recommendation dataset, and in Chapter 7, we implement more sophisticated neural rankers using DCN-V2, DLRM, and MMoE. Real-world recommender systems optimize for multiple objectives simultaneously. A music streaming service wants to maximize plays but also likes, minimize skips, and ensure diversity. Architectures like MMoE (Ma et al., 2018) that share expert networks across multiple prediction tasks (e.g., predict click, predict purchase, predict watch time simultaneously). In Chapter 7, we implement MMoE predicting five tasks simultaneously on the Yambda dataset.

Mixture of Experts (MoE) is an increasingly important pattern for LLMs and readers are encouraged to dive deeper than what this book covers.

Group 4: Sequence Models

Though sequence models are often used in ranking stages, they are mentioned separately due to their strengths in understanding user sequence history. Sequence models are a class of models that better capture the temporal dynamic of user interest by encoding the user's recent interaction history as an ordered sequence. GRU4Rec (Hidasi et al., 2016) applies RNNs to session data. SASRec and BERT4Rec use transformer architectures for sequential recommendation.

These models are particularly powerful in domains with strong session-level intent signals such as e-commerce browsing sessions, music listening sessions, news reading sessions. In Chapter 11, we build sequence-based user embeddings and compare them against simpler aggregation-based approaches on both the Amazon KDD and MIND datasets. Sequence models are also a stepping stone into understanding more recent techniques of generative recommenders which includes HSTU, Semantic IDs, etc.

Group 5: Multi-Objective and Decision Policies

- **Value Functions:** Weighted combinations of multiple model outputs that produce a single score reflecting business priorities. In Chapter 8, we build a complete value function that blends listen probability, like probability, engagement ratio, and dislike probability with tunable business weights.
- **Contextual Bandits and Exploration Policies:** Methods like Thompson Sampling and epsilon-greedy that balance exploitation (showing what the model thinks is best) with exploration (showing items with high uncertainty to gather more information). We cover these in Chapter 16. MABs are often used as meta policies that decide when to use what algorithm.

More recently we have seen the rise of LLM-based recommenders that are beyond the grouping indicated here and are covered in Chapter 12.

1.4 The Personalization Spectrum

Personalization, the adaptation of a product experience to an individual user, exists on a sliding scale. The algorithmic approach chosen when building a recommender system depends heavily on two factors: how much the system knows about the user and how much the product experience allows the user to express their intent.

At one end of the spectrum, you have anonymous or new users with no interaction history. If the UX is such that users don't provide you with their profile information, you are limited to non-personalized approaches: popularity-based ranking, trending items, or content-based recommendations using only the item's metadata and perhaps coarse contextual signals like geography or time of day.

As you accumulate more user-specific data especially clicks, impressions, and conversions collaborative signals become viable. Session-based models can begin to infer intent from just a handful of interactions within a single visit.

At the other end, for users with rich interaction histories, you can deploy sophisticated approaches: learned user embeddings, multi-task ranking models, and personalized retrieval. The richest personalization happens when the product experience itself

encourages interaction: rating systems, explicit preferences, wishlists, and rich clickstream data all feed the recommendation engine.

Figure 1.1. The Personalization Spectrum

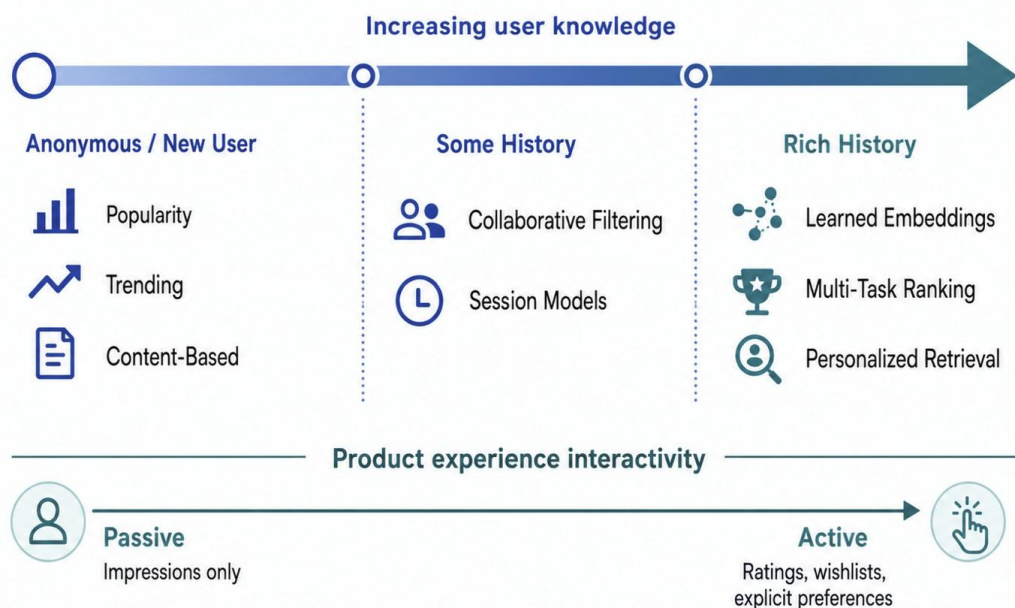


Figure 1.1: The personalization spectrum across increasing user knowledge and product interactivity. As user history becomes richer, systems can progress from non-personalized methods to collaborative, session-based, and learned representation approaches. Product experiences also range from passive observation to active collection of ratings, wishlists, and explicit preferences. Image Credits - Author

For RecSys engineers, this spectrum is a practical planning tool. When scoping a new personalization initiative, one of the first questions to ask is: where on this spectrum are our users, and where on this spectrum does our product experience allow them to be?

1.5 How RecSys Differs from Traditional Data Science

If you are starting in this domain coming in with a background of training classical ML models, building a recommender system can initially feel like a familiar exercise. You have features, labels, and a model that predicts outcomes. But there are fundamental differences that catch even experienced ML engineers off guard - they change how you design systems, how you evaluate models, and how you organize your engineering team.

Recommender systems sit at the intersection of engineering, artificial intelligence, machine learning, deep learning, and data science. Building an AI-powered personalization system requires ML models for retrieval and ranking, AI-level system

design, data science rigor for evaluation & experimentation and finally the engineering heavy lifting to make this work at scale at low latency.

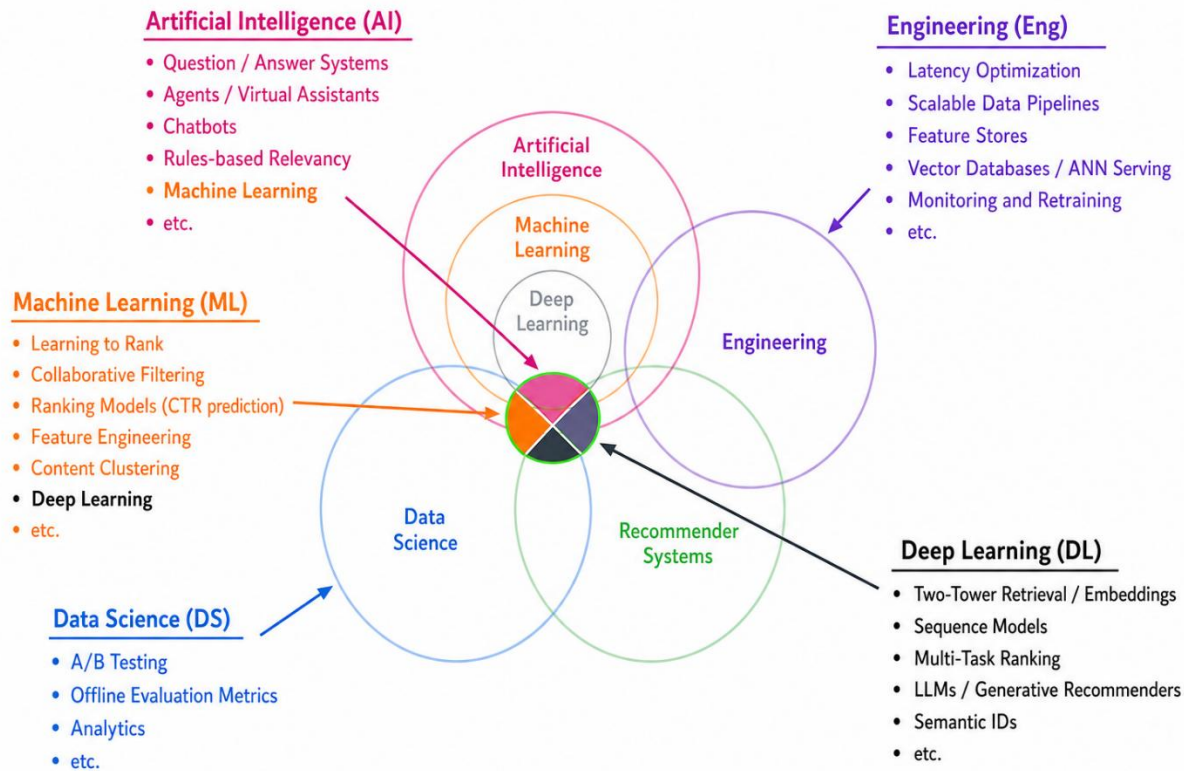


Figure 1.2: Representative techniques from each discipline that contribute to AI-powered personalization. Machine learning contributes learning to rank and collaborative filtering, deep learning contributes embedding-based retrieval and generative recommenders, data science contributes A/B testing and offline evaluation, and engineering contributes the latency optimization, data pipelines, and serving infrastructure that make these models usable in production. Adapted from the AI-powered search framing of Grainger et al. (2024)

Large Candidate Spaces and the Latency-Accuracy Tradeoff

A conventional classifier usually chooses among a fixed set of labels. A recommender system may instead choose from millions of items in a catalog that changes as items are added, removed, or become temporarily ineligible. The candidate set, the collection of items available for ranking at a particular stage, is therefore part of the ML problem. Retrieval must preserve relevant items before a ranker can score them, because even an excellent ranker cannot recover an item that the retrieval stage omitted. This large output space motivates the retrieval-and-ranking architecture introduced in Chapter 2 and developed in Chapter 5.

In many ML applications, inference latency is a secondary concern. It is acceptable for an offline model to take minutes or hours to score customers who are likely to click on an email or a batch forecasting model that predicts financial revenue every month. A recommender system that takes more than 200ms to populate a feed is not okay because users will notice the lag and will drop to go to a competitor's platform.

This constraint, typically in the low few 100 ms end-to-end, is what drives the multi-stage architecture that dominates production RecSys: a fast but approximate retrieval stage that narrows millions of candidates to hundreds, followed by a slower but more accurate ranking stage, often followed by re-ranking and business rule application stages. Each stage trades off accuracy for latency. We dissect this architecture in Chapter 2 and implement each stage in Chapters 5 through 8.

Feedback Loops

In many ML systems, the model predictions do not affect its future instances of the training data. For example, a weather model's prediction does not influence the weather. A recommender system is different: the model's predictions determine what users see, and what users see determines what they click, and what they click becomes tomorrow's training data.

This creates feedback loops that can be virtuous (the model learns to surface increasingly relevant content) or vicious (the model reinforces its own biases, creating filter bubbles where users are never exposed to content outside their predicted preferences). If the model shows cat videos to a user, then the user will likely click on it at some point just from sheer happenstance, even if they are not originally interested in it. This click is then treated as a positive signal and the model subsequently shows an increasing number of cat videos.

Managing these feedback loops is a core challenge in RecSys that has no real analogue in traditional ML. We discuss mitigation strategies in Chapters 2 and 3, and exploration-exploitation tradeoffs in Chapter 16.

Training Data Is Selectively Observed

Supervised learning often assumes that labels are observed across a representative sample of examples. Recommender systems observe feedback only for items that an existing policy chose to expose. An item that was never shown has no behavioral label, while a displayed item that received no click may have been ignored, overlooked, or genuinely disliked. The resulting data is missing not at random because the probability of observing feedback depends on the serving policy itself (Schnabel et al., 2016). The previous ranking policy also determines where an exposed item appears. Items near the top of a page generally receive more attention, creating position bias, while the

policy's choice of which items to show creates exposure bias. Training directly on these logs can cause a successor model to reproduce the predecessor's choices rather than learn users' underlying preferences (Joachims et al., 2017). Chapter 3 develops methods for constructing and evaluating training data under these biases.

The Correctness Ambiguity

In classification problems you often encounter in toy datasets like the titanic, you are given a clear ground truth. The ship's passenger either died or survived. In recommender systems, ground truth is fundamentally ambiguous.

A user clicked on a product, does that mean they liked it? They might have been curious, misled by a clickbait thumbnail, or simply scrolling past. Let's say that a user watched the first 10 minutes of a 90-minute movie and this watch history is logged by the data collection system. When preparing the training dataset, should the ML engineer consider that 10 minute watch as a positive label because the user was interested enough to start? Or treat it as a negative signal since the user didn't complete it? What if the user came back the next day and finished the movie?

Selective observation determines which outcomes enter the log; correctness ambiguity determines what an observed behavior means. Both problems complicate label construction, but they require different reasoning. Exposure and position information help determine whether an outcome was observable, while domain knowledge helps interpret the behavior after it occurred.

Every label in RecSys is a noisy proxy for unobservable user preference, some more so on the other. The engineering challenge is constructing label definitions, feature representations, and evaluation metrics that are good enough proxies, while remaining aware of their limitations. We will cover these and other data processing/ training data creation practicalities in depth in Chapter 3.

A recommender system also produces an ordered slate, meaning the complete list of items shown together, rather than a collection of independent predictions. A slate containing individually relevant items can still provide a poor experience if every item is nearly identical or if the ordering violates product constraints. RecSys evaluation and optimization must therefore consider list-level properties such as diversity, novelty, coverage, and business objectives alongside item-level prediction quality. Chapters 3 and 8 examine these considerations in greater depth.

Infrastructure as a First-Class Citizen

In traditional data science, the infrastructure such as the data pipelines, feature stores, serving systems, is often an afterthought. The data scientist trains a model in a Jupyter notebook and “throws it over the wall” to an engineering team for deployment.

In RecSys, the infrastructure is a prerequisite when building the system. You need a feature store to serve features at low latency without train-serve skew. You need vector databases for approximate nearest neighbor search in retrieval. You need streaming pipelines to compute real-time features from clickstream data. You need an experimentation platform to run A/B tests at scale. And you need to know the amount of compute available to know ahead of time what size of the model to even attempt building.

The quality of your infrastructure directly bounds the quality of your models. A brilliant ranking model is useless if your feature pipeline has a 24-hour lag on critical real-time features. We cover scalable data pipelines and feature management in Chapter 4, and deployment infrastructure in Chapters 13 and 14.

1.6 Business Impact of Personalization

The business case for recommendation systems is well established across multiple consumer-facing domain:

- **Streaming Media:** Netflix has reported that its recommender system influenced about 80 percent of hours streamed, while newer research confirms that personalization affects viewing decisions (Gomez-Urbe and Hunt, 2015; Zielnicki et al., 2026). YouTube reported that recommendations drove more than 70 percent of watch time (Nieva, 2018).
- **E-Commerce:** The U.S. Census Bureau estimated \$326.7 billion in U.S. retail e-commerce sales during the first quarter of 2026, accounting for 16.9 percent of total retail sales (U.S. Census Bureau, 2026). Recommender systems shape product discovery across this large market, and patterns such as "Customers who bought X also bought Y" have become foundational to online retail.
- **Social Media:** When Instagram shifted from a chronological feed to an algorithmic one, it saw increases in both time spent and engagement metrics. Meta's feed ranking algorithm is perhaps the highest-traffic recommender system in the world.
- **Advertising:** Personalized ad targeting is a multi-hundred-billion dollar industry. In Chapter 9, we explore the specialized modeling techniques (ESMM, federated learning) that address the unique challenges of ad recommendation, including extreme class imbalance and selection bias.
- **Music and Audio:** Spotify's Discover Weekly playlist, powered by collaborative filtering and content analysis, has become one of the platform's most beloved features and a significant driver of user retention.

What these examples share is a common pattern: personalization drives engagement, engagement drives retention, and retention drives revenue. For organizations considering investment in RecSys, the question is rarely “should we personalize?” but rather “how sophisticated does our personalization need to be, given data assets and user base, engineering and budget considerations?”

1.7 How This Book Is Organized

This book aims to cover most aspects of a production recommender system, from the broadest system design decisions down to the specific modeling techniques and operational practices that make them work in production.

Chapters 1–4 lay the conceptual foundation. Chapter 2 introduces multi-stage architecture and provides a comprehensive framework for thinking through RecSys system design. Chapter 3 covers training data collection, labeling strategies, and offline evaluation: the data engineering that makes or breaks your models. Chapter 4 addresses scalable data pipelines and feature management, including feature stores.

Chapters 5–9 cover the core modeling stages. Chapter 5 implements retrieval using two-tower models on the SIGIR E-commerce (Coveo) dataset. Chapter 6 builds ranking models with XGBoost on the Yandex Yambda dataset. Chapter 7 extends ranking to neural architectures (DCN-V2, DLRM, MMoE). Chapter 8 introduces value functions and diversity optimization, completing the multi-stage pipeline. Chapter 9 adapts these techniques for adtech use cases using the Ali-CCP and FedAds datasets.

Chapters 10–11 cover reusable item and user representations. Chapter 10 is a deep dive into item embeddings, comparing pre-trained text encoders, collaborative embeddings, contrastive fine-tuning, and multi-modal fusion across four datasets (MIND, Amazon KDD, Music4all, IJCAI). Chapter 11 builds user embeddings using aggregation, sequence-based, and graph-based (LightGCN) approaches.

Chapters 12–16 cover advanced topics and production operations. Chapter 12 explores how large language models can enhance recommender systems. Modern topics like Semantic IDs are covered with worked examples. Chapters 13 and 14 address online deployment and latency optimization. Chapter 15 covers monitoring and retraining - the operational practices that keep RecSys systems healthy in production. Chapter 16 discusses experimentation and bandits, including how to run effective A/B tests and balance exploration with exploitation.

Each chapter from 5 onward includes hands-on code tutorials with real-world datasets, so you can build and evaluate the models discussed. Simple baselines are deliberately introduced because they serve as a quick path to production and a way to debug

production pipeline. RecSys engineers should internalize these techniques before reaching for more sophisticated model in the toolbox.

1.8 Summary

Recommender systems are the engine behind the personalized digital experiences we interact with daily. They combine collaborative signals, content features, contextual information, and learned representations to score and rank items for individual users under strict latency constraints.

In this chapter, we introduced the core scoring function $f(\text{user}, \text{item}, \text{context})$, traced the evolution from collaborative and content-based filtering to modern hybrid approaches, and organized the landscape of modeling approaches into five practical groups: similarity methods, retrieval engines, ranking models, sequence models, and multi-objective/decision policies.

We also highlighted several ways that RecSys engineering differs from traditional data science: large and dynamic candidate spaces, strict latency constraints, policy-dependent training data, feedback loops, ambiguous labels, slate-level objectives, and the critical role of infrastructure. These themes will recur throughout the book.

In the next chapter, we deconstruct the multi-stage architecture that production recommender systems use to navigate these challenges, and develop a comprehensive framework for system design.

References

- Burges, C., Shaked, T., Renshaw, E., Lazier, A., Deeds, M., Hamilton, N., and Hullender, G. (2005). Learning to Rank Using Gradient Descent. ICML.
- Covington, P., Adams, J., and Sargin, E. (2016). Deep Neural Networks for YouTube Recommendations. Proceedings of the 10th ACM Conference on Recommender Systems. ACM.
- Goldberg, D., Nichols, D., Oki, B. M., and Terry, D. (1992). Using Collaborative Filtering to Weave an Information Tapestry. Communications of the ACM, 35(12).
- Gomez-Uribe, C. A., and Hunt, N. (2015). The Netflix Recommender System: Algorithms, Business Value, and Innovation. ACM Transactions on Management Information Systems, 6(4).
- Hidasi, B., Karatzoglou, A., Baltrunas, L., and Tikk, D. (2016). Session-Based Recommendations with Recurrent Neural Networks. ICLR.
- Joachims, T., Swaminathan, A., and Schnabel, T. (2017). Unbiased Learning-to-Rank with Biased Feedback. WSDM.
- Linden, G., Smith, B., and York, J. (2003). Amazon.com Recommendations: Item-to-Item Collaborative Filtering. IEEE Internet Computing, 7(1).
- Ma, J., Zhao, Z., Yi, X., Chen, J., Hong, L., and Chi, E. H. (2018). Modeling Task Relationships in Multi-Task Learning with Multi-Gate Mixture-of-Experts. KDD.
- Nieva, R. (2018). YouTube's AI Recommendations Drive 70 Percent of Viewing. CNET.
- Pandora. (2022). What Is the Music Genome Project? Pandora Community.
- Resnick, P., Iacovou, N., Suchak, M., Bergstrom, P., and Riedl, J. (1994). GroupLens: An Open Architecture for Collaborative Filtering of Netnews. Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work. ACM.
- Schnabel, T., Swaminathan, A., Singh, A., Chandak, N., and Joachims, T. (2016). Recommendations as Treatments: Debiasing Learning and Evaluation. ICML.
- U.S. Census Bureau. (2026). Quarterly Retail E-Commerce Sales, First Quarter 2026. U.S. Department of Commerce.
- Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W. L., and Leskovec, J. (2018). Graph Convolutional Neural Networks for Web-Scale Recommender Systems. KDD.

- Zielnicki, K., Aridor, G., Bibaut, A., Tran, A., Chou, W., and Kallus, N. (2026). The Value of Personalized Recommendations: Evidence from Netflix. *International Journal of Industrial Organization*.

Knowledge Graphs vs. Graph Neural Networks: Throughout this book, when we discuss graph-based methods (such as LightGCN in Chapter 11 or PinSage for retrieval), we are referring to Graph Neural Networks (GNNs) that learn node embeddings from the structure of user-item interaction graphs. These should **not** be conflated with Knowledge Graphs, which are structured databases of entities and their semantic relationships (e.g., “The Dark Knight” → directed_by → “Christopher Nolan”). Knowledge Graphs are a separate technology from the GNN-based embedding methods and are beyond the scope of this book.